Лекция №3.
MPI. Продвинуто.



Григорьев Василий Васильевич

2023



Как стать соучастником

https://www.mpi-forum.org

Участники сообщества продолжают встречаться каждые 3 месяца для решения дальнейшей судьбы MPI.

Вы тоже можете стать частью этого движения. Вход бесплатный.

Выводы

- Параллелизм очень важен в современных реалиях для достижения наилучшей производительности с современным железом;
- MPI является стандартом индустрии для параллельного программирования (существует большое количество реализаций MPI – коммерческий и бесплатный; каждая система в мире поддерживает MPI);
- Дает пользователю явный контроль над менеджментом данных;
- Широко используется многими научными приложениями в большим успехом.

Модель программирования

- Программа состоит из P параллельных процессов, которые порождаются при запуске программы (MPI 1) или могут быть динамически созданы во время выполнения (MPI 2);
- Каждый процесс имеет уникальный идентификатор [0, P – 1] и изолированное адресное пространство (SPMD);
- Процессы взаимодействуют путем передачи сообщений (message passing)
- Процессы могут образовывать группы для реализации коллективных операций.

Понятие коммуникатора

- Коммуникатор (communicator) – группа процессов, образующая логическую область для выполнения коллективных операций между процессами;
- В рамках коммуникатора процессы имеют номера: 0, 1, …, P – 1;
- Все MPI-сообщения должны быть связаны с определенным коммуникатором;
- Коммуникатор MPI_COMM_WORLD включает все процессы.

Поскольку мы уже можем писать параллельные программы

Пора вникнуть в нюансы, которые могут помочь в будущем, в ваших научных исследованиях и т.п.

Показатели эффективности параллельных алгоритмов

- Коэффициент ускорения (SpeedUp)
- Коэффициент эффективности (Efficiency)
- Коэффициент накладных расходов
- Показатель равномерности загруженности параллельных ветвей (процессов, потоков)

Коэффициент ускорения (Speedup)

- Введем обозначения:
  - ❑ $\boldsymbol{T(n)}$ – время выполнения последовательной программы (sequential program)
  - ❑ $\boldsymbol{Tp(n)}$ – время выполнения параллельной программы (parallel program) на p процессорах
- **Коэффициент *Sp(n)* ускорения** параллельной программ (Speedup):

$$\boldsymbol{Sp(n) = T(n) / Tp(n)}$$

- Коэффициент ускорения $Sp(n)$ показывает во сколько раз параллельная программа выполняется на *p* процессах быстрее последовательной программы при обработке одних и тех же входных данных размера n
- Как правило

$$\boldsymbol{Sp(n) \leq p}$$

- ***Цель распараллеливания – достичь линейного ускорения на максимально большом числе процессоров***

Коэффициент ускорения (Speedup)

- Какое время брать за время выполнения последовательной программы?

Время лучшего известного алгоритма (в смысле вычислительной сложности)?

Время лучшего теоретически возможного алгоритма?

- Что считать временем выполнения $\boldsymbol{Tp(n)}$ параллельной программы?

Среднее время выполнения потоков программы?

Время выполнения потока, завершившего работу первым?

Время выполнения потока, завершившего работу последним?

Коэффициент ускорения (Speedup)

- Какое время брать за время выполнения последовательной программы?

  Время лучшего известного алгоритма или время алгоритма, который подвергается распараллеливанию

- Что считать временем выполнения $\boldsymbol{Tp(n)}$ параллельной программы?

  Время выполнения потока, завершившего работу последним

Коэффициент относительного ускорения (Rel. speedup)

- Коэффициент относительного ускорения (Relative speedup) – отношения времени выполнения параллельной программы на k процессорах к времени её выполнения на p процессорах (k < p)

$$\boldsymbol{SRel(k, p, n) = Tk(n) / Tp(n)}$$

- Коэффициент эффективности (Efficiency) параллельной программы

$$\boldsymbol{Ep(n) = Sp(n)/p = T(n)/(p*Tp(n)) \in [0, 1]}$$

- Коэффициент накладных расходов (Overhead)
- $\boldsymbol{\varepsilon\ (p, n) = TSync(p, n) / TComp(p,n) = (TTotal(p,n) - TComp(p,n)) / TComp(p,n)}$
- $\boldsymbol{TSync(p,n)}$ – время создания, синхронизации и взаимодействия p потоков
- $\boldsymbol{TComp(p,n)}$ – время вычислений в каждом из p потоков

Виды масштабируемости программ

Масштабируемость параллельной программы (scalability) – характеристика программы, показывающая как изменяются ее показатели производительности при варьировании числа параллельных процессов на конкретной ВС

Строгая/сильная масштабируемость (strong scaling) – зависимость коэффициента ускорения от числа p процессов при фиксированном размере n входных данных (n = const)

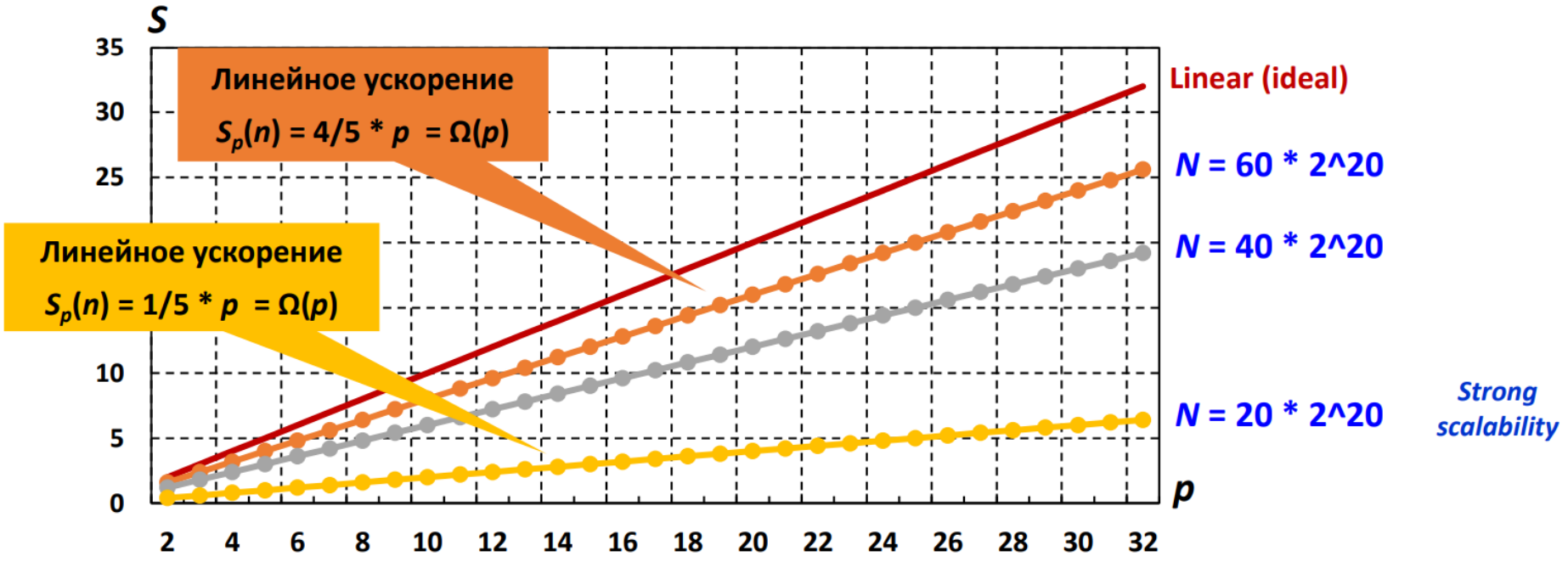
- ❑ Показывает как растут накладные расходы с увеличением p
- ❑ Цель – минимизировать время решения задачи фиксированного размера

Слабая масштабируемость (weak scaling) – зависимость коэффициента ускорения параллельной программы от числа процессов при фиксированном размере входных данных на один процессор (n / p = const)

- ❑ Цель – решить задачу наибольшего размера на ВС

Параллельная программа (алгоритм) коэффициент ускорения, которой линейной растет с увеличением p называется линейно масштабируемой или просто масштабируемой (scalable)

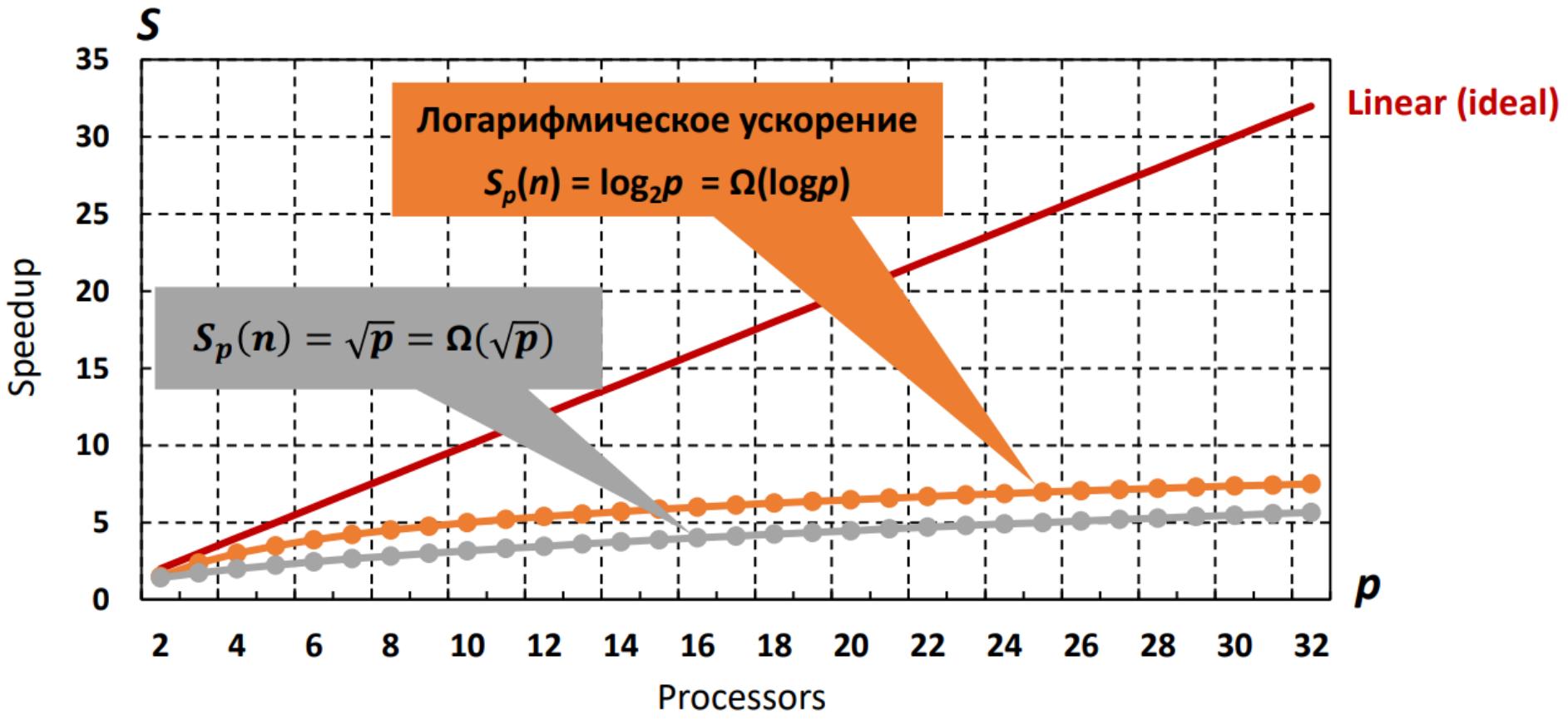
Коэффициент ускорения (SpeedUp)



Зависимость коэффициента ускорения *S*
параллельного алгоритма X от количества *p* процессоров

- Ускорение программы может расти с увеличением размера входных данных
- Время вычислений превосходит накладные расходы на взаимодействия потоков (управление потоками, синхронизацию, обмен сообщениями, …)

Коэффициент ускорения (SpeedUp)



Зависимость коэффициента ускорения *S*
параллельных алгоритмов Y и Z от количества *p* процессоров

Суперлинейное ускорение (superlinear speedup)

Параллельная программа может характеризоваться суперлинейным ускорением (superlinear speedup) – коэффициент ускорения Sp(n) принимает значение больше p

$$\boldsymbol{Sp(n) > p}$$

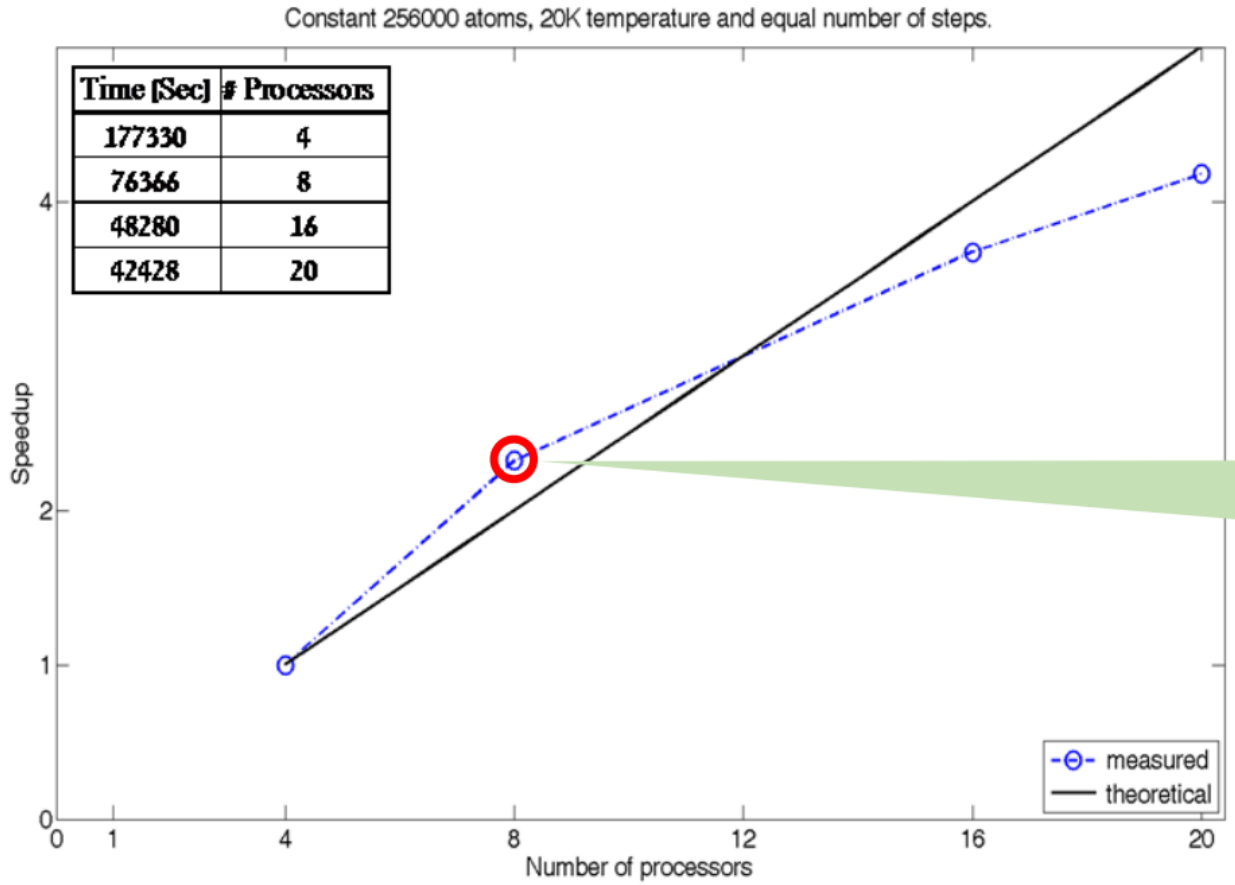
Причина: иерархическая организация памяти: Cache – RAM – Local disk (HDD/SSD) – Network storage

Последовательная программ выполняется на одном процессоре и обрабатывает данные размера n

Параллельная программа имеет p потоков на p процессорах, каждый поток работает со своей частью данных, большая часть которых может попасть в кеш-память, в результате в каждом потоке сокращается время доступа к данным

Тот же самый эффект можно наблюдать имя два уровня иерархической памяти: диск – память

Суперлинейное ускорение (superlinear speedup)



Superlinear speedup

$$S_8 = \frac{T_4}{T_8} = 2.32$$

Parallel Molecular Dynamic Simulation

MPI, Spatial decomposition; Cluster nodes: 2 x AMD Opteron Dual Core; InfiniBand network

Равномерность распределения вычислений

- **По какому показателю оценивать равномерность времени выполнения потоков/процессов параллельной программы?**
- Известно время выполнения потоков t_0, t_1, …, t_p
- Коэффициент *V* вариации

$$\boldsymbol{V = \frac{\sigma[t_i]}{\mu[t_i]}}$$

- Отношение min/max

$$\boldsymbol{M = \frac{\min\{t_i\}}{\max\{t_i\}}}$$

- Jain's fairness index

$$\boldsymbol{f = \frac{\left(\sum_{i=0}^{p-1} t_i\right)^2}{n \sum_{i=0}^{p-1} t_i^2} \in [0, 1]}$$

Закон Дж. Амдала (Amdahl's law)

- Пусть имеется последовательная программа с временем выполнения $T(n)$
- Обозначим:
  - $r \in [0, 1]$ – часть программы, которая может быть распараллелена (perfectly parallelized)
  - $s = 1 - r$ – часть программы, которая не может быть распараллелена (purely sequential)
- Время выполнения параллельной программы на p процессорах (время каждого потока) складывается из последовательной части s и параллельной r:

$$T_p(n) = T(n)s + \frac{T(n)}{p}r$$

- Вычислим значение коэффициент ускорения (по определению)

$$S_p(n) = \frac{T(n)}{T_p(n)} = \frac{T(n)}{T(n)s + \frac{T(n)}{p}r} = \frac{1}{s + \frac{r}{p}} = \frac{1}{(1 - r) + \frac{r}{p}}$$

- Полученная формула по значениям r и s позволяет оценить максимальное ускорение



Закон Дж. Амдала (Amdahl's law)

- Пусть имеется последовательная программа с временем выполнения $T(n)$
- Обозначим:
  - $\boldsymbol{r} \in [\mathbf{0}, \mathbf{1}]$ – часть программы, которая может быть распараллелена (perfectly parallelized)
  - $\boldsymbol{s} = \mathbf{1} - \boldsymbol{r}$ – часть программы, которая не может быть распараллелена (purely sequential)
- **Закон Дж. Амдала (Gene Amdahl, 1967) [1]:**

  Максимальное ускорение S_p программы на p процессорах равняется

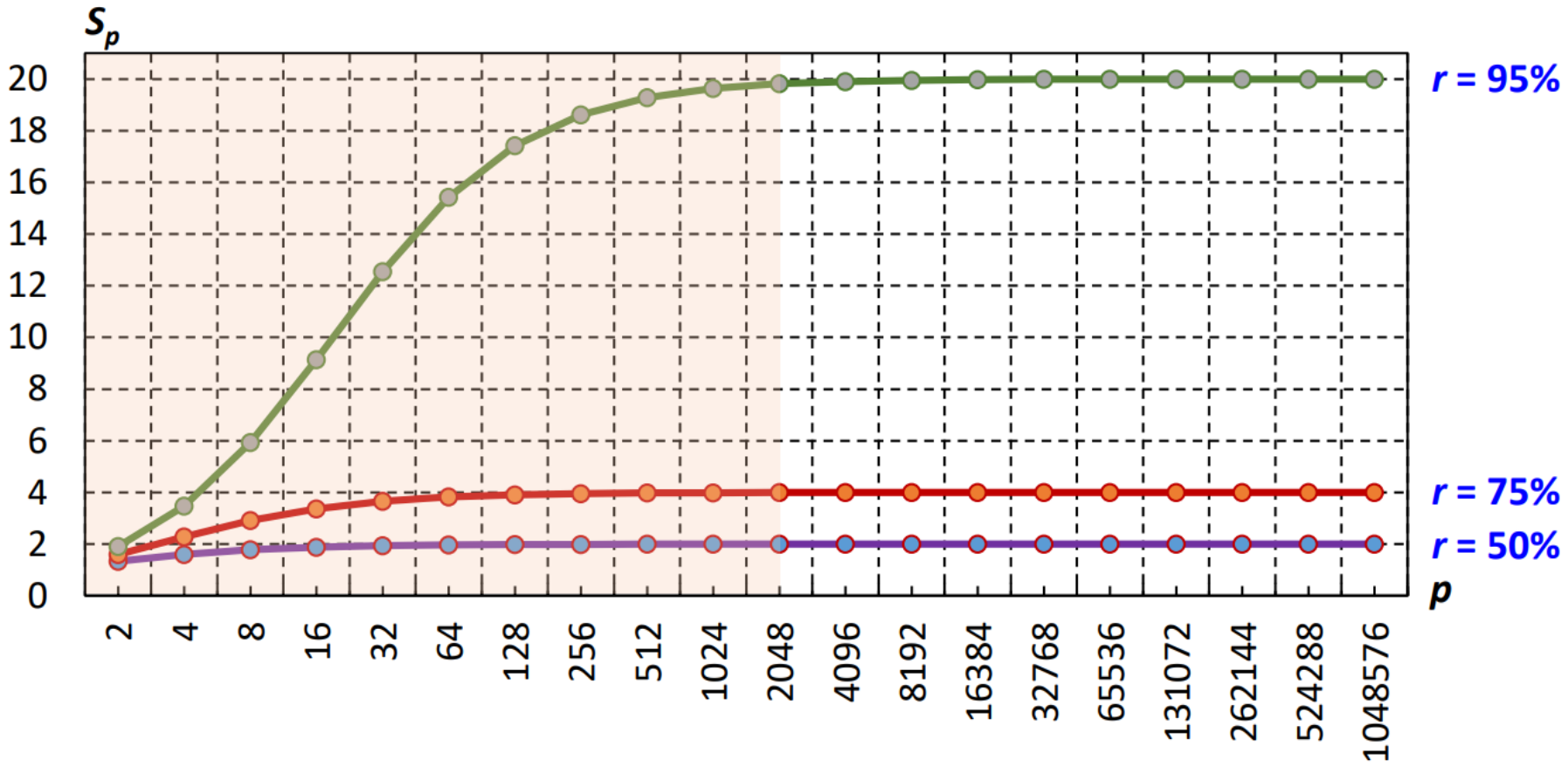
$$\boldsymbol{S_p} = \frac{\mathbf{1}}{(\mathbf{1} - \boldsymbol{r}) + \frac{\boldsymbol{r}}{\boldsymbol{p}}}$$

$$\boldsymbol{S_\infty} = \lim_{p \to \infty} S_p = \lim_{p \to \infty} \frac{1}{(1 - r) + \frac{r}{p}} = \frac{1}{1 - r} = \frac{\mathbf{1}}{\boldsymbol{s}}$$



❑ Amdahl Gene. **Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities** // AFIPS Conference Proceedings, 1967, pp. 483-485, http://www-inst.eecs.berkeley.edu/~n252/paper/Amdahl.pdf

Закон Дж. Амдала (Amdahl's law)



Зависимость коэффициента S_p ускорения параллельной программы от количества p процессоров

Допущения закона Дж. Амдала (Amdahl's law)

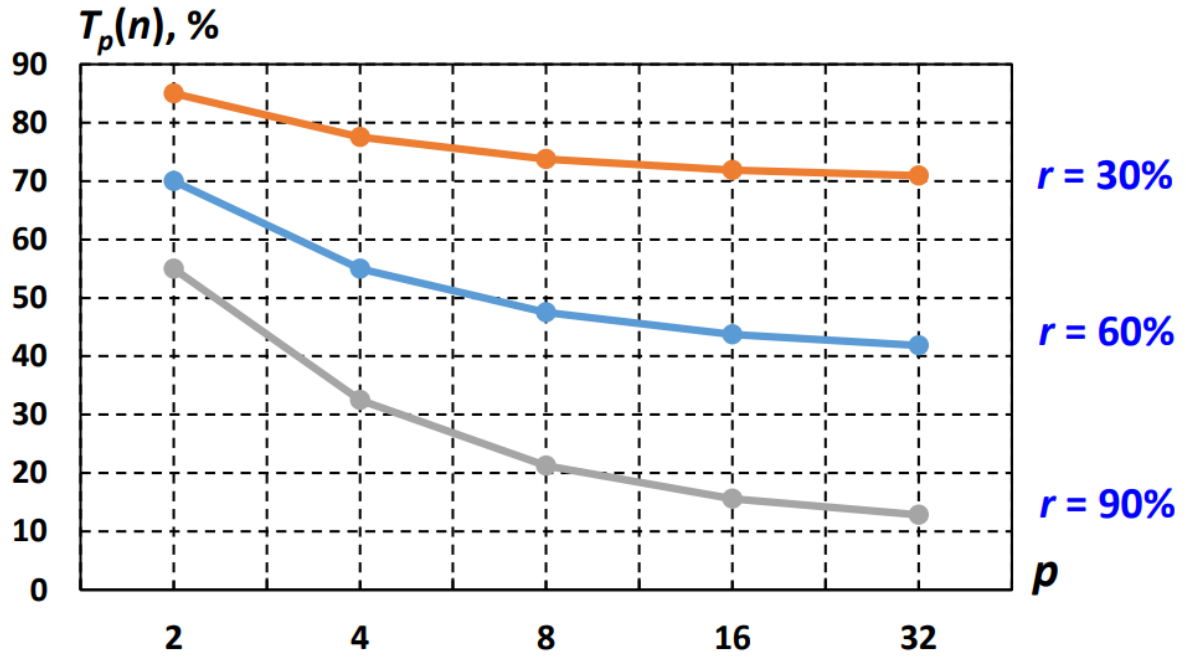
- **Последовательный алгоритм является наиболее оптимальным способом решения задачи**
- Возможны ситуации когда параллельная программа (алгоритм) эффективнее решает задачу (может эффективнее использовать кеш-память, конвейер, SIMD-инструкции, …)
- **Время выполнения параллельной программы оценивается через время выполнения последовательной**, однако потоки параллельной программы могут выполнятся эффективнее

$$T_p(n) = T(n)s + \frac{T(n)}{p}r, \quad \text{на практике возможна ситуация } \frac{T(n)}{p} > T_p(n)$$

- **Ускорение $S_p(n)$ оценивается для фиксированного размера n данных при любых значениях p**
- В реальности при увеличении числа используемых процессоров размер n входных данных также увеличивают, так как может быть доступно больше памяти

Закон Дж. Амдала (Amdahl's law)

- **На что потратить ресурсы – на увеличение доли *r* параллельной части в программе или увеличение числа процессоров, на которых запускается программа?**



Зависимость времени $T_p(n)$ выполнения параллельной программы от количества *p* процессоров и доли *r* распараллеленного кода (время в % от времени $T_1(n)$)

Закон Дж. Амдала (Amdahl's law)

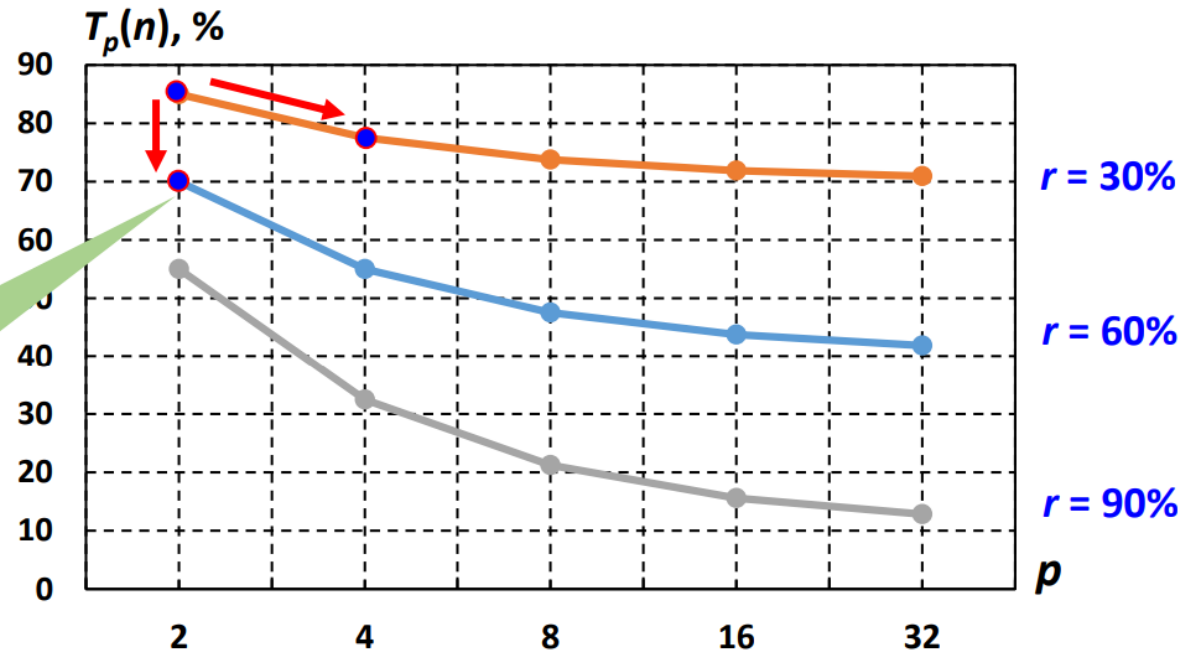
- **На что потратить ресурсы – на увеличение доли *r* параллельной части в программе или увеличение числа процессоров, на которых запускается программа?**

Увеличили число процессоров с 2-х до 4-х (программу не меняли)

Время выполнения сократилось с 85% до 77,5%

Увеличим в 2 раза долю параллельного кода

Время выполнения сократилось с 85% до 70%



Зависимость времени $T_p(n)$ выполнения параллельной программы от количества *p* процессоров и доли *r* распараллеленного кода (время в % от времени $T_1(n)$)

Закон Густафсона-Барсиса

- Пусть имеется последовательная программа с временем выполнения $T(n)$
- Обозначим $s \in [0, 1]$ – часть параллельной программы, которая выполняется последовательно (purely sequential)
- **Закон Густафсона-Барсиса (Gustafson–Barsis' law) [1]:**

  Масштабируемое ускорение S_p программы на p процессорах равняется

$$S_p = p - s(p - 1)$$

- **Обоснование:** пусть a – время последовательной части, b – время параллельной части

$$T_p(n) = a + b, \qquad T(n) = a + pb$$

$$s = a/(a + b), \qquad S_p(n) = s + p(1 - s) = p - s(p - 1)$$

- **Время выполнения последовательной программы выражается через время выполнения параллельной**

❑ **Reevaluating Amdahl's Law**, John L. Gustafson, Communications of the ACM 31(5), 1988. pp. 532-533 // http://www.scl.ameslab.gov/Publications/Gus/AmdahlsLaw/Amdahls.html

Спасибо за внимание!

